

Mainflux				Things	Channels	Logout
Id	Name	Type	Payload			
1	MyAPP1	app				
2	MyDev1	device				
+						

Figure 3: Dashflux home page

- The most likely problems to occur are port conflicts. If any of your applications is using a port that is needed to run Mainflux services, then you need to free that port or modify the default port from *docker-compose.yml*.
- Another common problem faced is failure to allow all the ports in the firewall configuration.

Using Mainflux

Mainflux is basically about messages, for which it provides a simple UI client known as Dashflux. As said earlier, to login for the first time you need to register by providing a valid user name and password. Once logged into the system, you can manage resources with the help of CRUD (create, read, update and delete) operations, and can define the access control policies. After login, the user is redirected to the home page (Figure 3).

You can see three buttons on the top right corner in the Dashflux home page. The *Things* button will navigate to the things dashboard, *Channels* to the channels dashboard and *Logout* will log you out. Clicking on the plus symbol will open the dialogue for adding a new thing or channel. Once a channel is created, the user can edit

it and connect or disconnect devices. Remember one thing can be connected to more than one channel, and vice versa. Mainflux uses Rest API — all the operations can be done by using various command line tools or any HTTP API development tool.

Figure 3 can be divided into two parts: core platform components and optional ones. Core platform components are NATS, protocol adaptors and *normalizer*. *Writer* and *database* are optional. Here, ‘thing’ represents an application or device that uses Mainflux for message exchange. As the need to use Mainflux is different for every company, the protocol adaptor is used to support various types of protocols like MQTT, WebSocket and HTTP. For every protocol used, there is a corresponding protocol adaptor. The job of the adaptor is to transform a protocol-specific message to a suitable format, i.e., Mainflux message. Then comes NATS, which is an open source scalable messaging system Mainflux uses for message exchange within a platform. Once

published to NATS, the message will be transferred to normaliser service. The job of *normaliser* is to convert the message to the SenML (Sensor Markup Language) format. *Normaliser* then forwards the converted message to NATS for further processing, to check whether it is in SenML or not. Only SenML formatted messages will be written in the database.

Security

Mainflux is a highly secured system. It has dedicated authentication and does not allow access to unauthorised devices and applications. All the messages and the network traffic are encrypted by the latest security standards such as TLS v1.3.

Mainflux is licensed under the Apache License 2.0. It is a lightweight and scalable platform. As all the services are deployed as a Docker container, the whole platform setup and installation takes only three steps. We can also customise whatever service or feature we like, and contribute to the Git repo if we wish to. 

 By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing.